

VOCAB CATALYST

IELTS & SAT

Vocab Catalyst

Software Engineering Technical Report

DOCUMENT TYPE	Technical Analysis Report
SUBJECT	Desktop Vocabulary Learning Application
PLATFORM	Python · CustomTkinter · SQLite · Google Gemini
DATE	7 June 2026
STATUS	Publication Ready

1		
2		
	Background	3
	Problem Statement	3
	Objectives	4
	Scope	4
3		
	Purpose and Target Users	4
	Main Functionalities	4
	Use Cases	5
4	System Architecture	
	Module Organization	5
	Component Interactions	6
	Data Flow	6
5	Technologies and Tools	
6	Functional Analysis	7
	Practice Generation	7
	Answer Submission and Feedback	7
	Hint Generation	8
	Mistake Bank and Analytics	8
	Relearning from Mistakes	8
	Study Roadmap and Calendar	9
	Reminder System	9
7		
8		
9		
	Algorithms	12
	Business Logic	13
	API Integrations	13
	Error Handling	13
	Configuration Management	14
10		
11		
12		

1	Challenges and Solutions	16
2		

1	Limitations	16
3		
1	Future Improvements	17
4		
1	Conclusion	17
5		
1	References	18
6		

SECTION 1

Abstract

*This report presents a technical analysis of the **IELTS & SAT Vocab Catalyst** repository — a desktop vocabulary-learning application implemented in Python. The system combines a graphical user interface built with CustomTkinter, a local SQLite persistence layer, and the Google Gemini API for dynamic exercise generation, feedback, and adaptive study planning. The application supports vocabulary learning for SAT and IELTS preparation through multiple-choice exercises, a mistake-tracking mechanism, AI-assisted explanations, and a calendar-based study roadmap.*

The codebase is organized around a modern desktop workflow. Responsibilities are separated into modular components for database access, data models, prompt construction, AI integration, desktop notifications, and UI rendering. The project demonstrates a practical integration of human-centered UI design, local data persistence, and generative AI in an educational desktop tool.

SECTION 2

Introduction

2.1 Background

Vocabulary acquisition is a central component of success in standardized English examinations such as the SAT and IELTS. Learners often need more than a static word list; they benefit from contextual practice, repeated exposure to previously missed words, and feedback that explains why an answer is correct or incorrect. Traditional flashcards and static quizzes are limited because they do not adapt to the learner's weaknesses or preserve the history of mistakes in a structured way.

This project addresses the educational need by implementing an AI-supported vocabulary trainer. Instead of serving fixed questions from a hard-coded dataset, the system requests Gemini to generate context-sensitive multiple-choice exercises on demand, and records mistakes, session scores, and a daily plan so that future practice can be tailored to the learner's performance.

2.2 Problem Statement

Learners preparing for SAT and IELTS vocabulary commonly face three recurring obstacles:

1. They encounter words in context but cannot reliably infer meaning from usage alone.
2. They forget previously missed words because incorrect answers are not systematically revisited.
3. Their study routine lacks a measurable daily schedule aligned to their weaknesses.

✓ Design Rationale

The codebase tackles all three issues by combining generation, review, analytics, and scheduling within a single cohesive desktop application.

2.3 Objectives

The primary objectives implemented in the codebase are:

- Generate vocabulary exercises dynamically using Google Gemini.
- Present exercises in a clear, responsive desktop GUI.
- Store incorrect responses in a local Mistake Bank via SQLite.
- Provide AI-generated hints and explanations in Vietnamese.
- Adapt difficulty based on recent session performance.
- Generate and persist a seven-day study roadmap.
- Support desktop reminder notifications on macOS.

2.4 Scope of the Project

The current repository focuses on a single-user desktop learning environment for SAT and IELTS vocabulary. Features within scope include multiple-choice vocabulary exercises, mistake logging and review, calendar-based planning, reminder notifications, local SQLite persistence, and AI integration through the Google Gemini API.

■ Out of Scope

The project does not include user authentication, multi-user synchronization, cloud storage, or a browser-based interface. A vocabulary import pipeline is also absent from the active runtime, despite a JSON vocabulary file existing in the repository.

SECTION 3

System Overview

3.1 Purpose and Target Users

The application is an AI-assisted vocabulary practice environment intended to help learners improve their command of high-frequency SAT and IELTS vocabulary. Its core function is to generate questions, evaluate answers, explain mistakes, and organize revision activities based on learner history.

The intended users are individual learners preparing for SAT or IELTS who prefer an interactive desktop tool over a static worksheet. The application is also suitable for teachers or mentors who wish to review a learner's mistake patterns locally, although the codebase is optimized for single-user operation.

3.2 Main Functionalities

Functionality	Description
Practice Generation	Requests Gemini to generate multiple-choice vocabulary exercises in one of several supported categories.
Hint Generation	Produces a short Vietnamese hint without revealing the correct answer.
Wrong-Answer Explanation	Generates a Vietnamese explanation after an incorrect response.
Mistake Bank	Stores each incorrect answer locally in SQLite for later review.
Session Logging	Records score, total questions, and difficulty after each completed session.
Adaptive Difficulty	Adjusts the stored difficulty value automatically based on session performance.
Roadmap Planning	Generates a 7-day study plan and stores it in the database.
Calendar Interface	Displays and manages planned tasks by date with completion checkboxes.
Desktop Reminders	Sends a daily macOS notification at a user-configured time.

Table 1. Core application functionalities.

3.3 Use Cases

The following primary use cases are supported by the implementation:

- A learner generates a short practice set for a selected exam, topic, and exercise type.
- The learner answers incorrectly and immediately receives an AI-generated explanation.
- The incorrect item is stored in the Mistake Bank and later seeds relearning exercises.
- The learner opens the calendar tab to review or check off daily tasks.
- The learner requests a new AI-generated 7-day roadmap based on recent performance.
- The learner enables a reminder and receives a daily notification at a chosen time.

SECTION 4

System Architecture

4.1 Architectural Overview

The project follows a modular desktop architecture with clear separation of concerns. The UI layer is implemented with CustomTkinter, the business logic is encapsulated in application methods and service classes, the AI integration is isolated in a Gemini wrapper, and persistence is managed through a lightweight SQLite helper module. The four-layer conceptual model is shown below.

■ User Interface Layer	CustomTkinter GUI — Tabs, widgets, event handlers
■ Application Orchestration	CatalystApp — workflow, threading, adaptive logic
■ Service Layer	GeminiService (AI) · SQLite helpers (db.py)
■ External Resources	Google Gemini API · Local SQLite Database

Figure 1. Four-layer conceptual architecture of Vocab Catalyst.

4.2 Module Organization

The active runtime modules are organized under the **vocab_catalyst/** package:

Module	Responsibility
app.py	Main GUI class and workflow orchestration
db.py	SQLite schema creation and data-access functions
gemini_service.py	Gemini client wrapper and JSON parsing utilities
models.py	Dataclasses for exercise and roadmap records
prompts.py	Prompt templates for all AI task types
constants.py	Supported exam names, topic lists, exercise types, difficulty levels
notify.py	macOS desktop notification helper
__init__.py	Package metadata and version string

Table 2. Active package modules and their responsibilities.

The repository root also contains several supporting files:

File	Role
catalyst_main.py	Main entry point — loads .env and launches the GUI
main.py	Minimal API smoke-test script (not part of the application stack)
vocab_quiz_app.py	Older monolithic quiz prototype (legacy)
data/catalyst.db	SQLite database used at runtime
data/vocabsat.json	Static vocabulary dataset with 790 entries (unused in active runtime)
RUN_GUIDE.md	Setup instructions — contains some stale references

Table 3. Supporting repository files outside the main package.

4.3 Component Interaction Sequence

The following sequence describes how the major components interact during a typical practice session:

1. **catalyst_main.py** loads environment variables from `.env` and calls **run_app()**.
2. **CatalystApp** initializes the SQLite database, the Gemini client, and the UI.
3. When the user requests practice, the UI gathers Mistake Bank data and builds a prompt.
4. **GeminiService** sends the prompt to Gemini and receives a JSON response.
5. The JSON is parsed into **ExercisePayload** or **RoadmapDay** dataclass objects.
6. The UI renders the items and stores learner outcomes back to SQLite.
7. Aggregation functions summarize mistakes and sessions to shape future prompts.
8. The background reminder thread checks settings and triggers macOS notifications.

4.4 Data Flow

■ Data Flow Summary

Input: user selections, answer choices, configuration values, and historical records from SQLite.

Processing: prompt construction, API invocation, JSON parsing, validation, adaptive logic, and database writes. **Output:** generated exercises, hints, explanations, roadmap entries, mistake

summaries, and desktop notifications.

Because the system relies on a large language model, the AI-generated content is not fully deterministic. However, the code imposes schema constraints and validates returned structures before they are accepted into the UI or the database.

SECTION 5

Technologies and Tools

Technology / Library	Usage in the Application
Python 3	Primary implementation language
CustomTkinter	Modern desktop GUI framework built on Tk
SQLite (stdlib)	Embedded single-file persistence layer
Google Gemini API (google-genai)	AI generation of exercises, hints, explanations, and roadmaps
threading (stdlib)	Background threads to keep the GUI responsive during network calls
dataclasses (stdlib)	Lightweight typed data models with built-in validation
json + re (stdlib)	Parsing and cleanup of Gemini JSON responses
os / pathlib (stdlib)	Configuration and cross-platform file path management
macOS osascript	Desktop notification delivery on Darwin platforms only

Table 4. Technologies and libraries used in Vocab Catalyst.

5.1 Dependency Review

The declared **requirements.txt** lists four packages. However, an audit of the active source files reveals that two of these packages are not used in the current runtime:

Package	Status
customtkinter	✓ Actively used
google-genai	✓ Actively used
numpy	✗ Declared but not used in active source files
sounddevice	✗ Declared but not used in active source files

Table 5. Dependency status audit.

■ Recommendation

The numpy and sounddevice packages should be removed from requirements.txt to reduce the installation footprint and avoid confusion about the application's capabilities.

SECTION 6

Functional Analysis

6.1 Practice Generation

Purpose

The practice module generates multiple-choice vocabulary exercises tailored to a selected exam, topic, exercise type, and difficulty level.

Workflow

The learner configures the following parameters before generating a session:

- **Exam:** SAT or IELTS
- **Topic:** one of the predefined topical categories
- **Exercise Type:** one of the supported question formats
- **Difficulty:** easy, medium, or hard
- **Item Count:** 1 to 5 questions

When the learner presses the generate button, the application retrieves recently mistaken words from the database, builds a prompt, and dispatches the request to Gemini in a background thread. The response is parsed into a queue of structured exercises.

Internal Implementation

`_on_generate_practice()` launches a worker thread. The method queries `mistake_word_frequencies()`, converts the top-frequency words into a seed list, and passes them into `GeminiService.generate_exercises()`. The response is validated by `parse_exercises_json()` and each item by `ExercisePayload.from_dict()`.

6.2 Answer Submission and Feedback

Workflow

When the learner submits an answer, the following sequence occurs:

1. The selected option is compared against the active exercise's correct index.
2. A correct answer advances the session to the next question.
3. An incorrect answer triggers `add_mistake()` to persist the error to SQLite.
4. Gemini is invoked asynchronously to explain the error in Vietnamese.
5. The explanation is appended to the feedback panel.

6. The session continues to the next question or ends if the queue is exhausted.

Internal Implementation

`_on_submit_practice()` compares the selected index with `ex.correct_index`. On failure it calls `add_mistake()` with full metadata: exam, topic, exercise type, target word, stem snippet, user choice, and correct choice. The Vietnamese explanation is generated asynchronously via `GeminiService.explain_wrong_vi()`.

6.3 Hint Generation

The hint function supports guided learning without revealing the correct answer. When the learner clicks the hint button during an active exercise, the application sends the question stem and answer choices to Gemini, requesting a short Vietnamese hint that explicitly avoids disclosing which option is correct. The prompt template `prompt_hint_vi()` instructs the model to avoid mentioning any answer label or answer content directly.

6.4 Mistake Bank and Review Analytics

The Mistake Bank preserves learner errors for later review and analytical pattern detection. Every incorrect answer automatically triggers `add_mistake()`. The tab lists recent mistakes in reverse chronological order and offers AI-powered analysis through `prompt_mistake_pattern_analysis()` and targeted relearning through `prompt_relearn_batch()`.

The function `mistake_word_frequencies()` computes the most frequently missed target words by grouping on the lowercase word and counting occurrences. These frequency data are then injected into prompts to personalize the review and diagnosis experience.

6.5 Relearning from Mistakes

This feature reconstructs practice sessions around previously missed vocabulary items. The application retrieves frequent mistake words and recent session summaries, then asks Gemini to create a mixed review batch using those weak words. If the learner has recently achieved perfect accuracy, the difficulty is optionally increased. If no mistakes exist in the bank, a local informational message is displayed and the feature exits gracefully.

6.6 Study Roadmap and Calendar

The roadmap module creates a seven-day study plan and displays it within a calendar interface. The learner selects a start date in the calendar tab and clicks the roadmap button. The application gathers weak-word data and recent performance metrics, sends them to Gemini, and expects exactly seven `RoadmapDay` entries in return.

■ Validation Rule

If Gemini returns a result that does not contain exactly seven day entries, the update is rejected and the user sees an error message. No partial roadmaps are written to the database. When a valid seven-day plan is received, `delete_plans_between()` removes any conflicting rows in the same date range, and `insert_daily_plans()` inserts the new records. Each row stores the date, exam, topic, exercise type, target count, completion flag, and an AI-generated explanatory note.

6.7 Reminder System

The reminder system provides a daily study nudge notification. The user configures a reminder time in HH:MM format and toggles the reminder flag from the `Nh■c nh■` tab. A background thread wakes up periodically, compares the current local time against the stored setting, and sends a macOS notification if the time matches and the reminder has not yet fired today.

The last firing date is stored in the `settings` table under the key `reminder_last_fired`, providing a debounce mechanism that prevents duplicate notifications on the same day. On non-Darwin operating systems, `desktop_notify()` returns without action.

SECTION 7

Database Design

7.1 Schema Overview

The active database is **data/catalyst.db**. It is initialized automatically on startup by **init_db()** using plain SQLite DDL statements. The schema is intentionally simple and largely denormalized to reduce complexity for a single-user desktop application.

Table	Purpose
mistakes	Stores individual incorrect answers for review
session_log	Stores session-level performance summaries
daily_plan	Stores daily study roadmap entries
settings	Stores key-value application configuration items

Table 6. Database tables and their purposes.

7.2 Table Structures

mistakes

Column	Type	Description
id	INTEGER PK	Auto-increment primary key
timestamp	TEXT	ISO-8601 datetime of the mistake
exam	TEXT	Exam type (SAT or IELTS)
topic	TEXT	Topical category
exercise_type	TEXT	Question format label
word	TEXT	Target vocabulary word
stem	TEXT	Snippet of the question stem
user_choice	TEXT	Option selected by the learner
correct_choice	TEXT	The correct answer option

Table 7. Schema of the mistakes table.

session_log

Column	Type	Description
id	INTEGER PK	Auto-increment primary key
session_date	TEXT	Date of the practice session
correct	INTEGER	Number of correctly answered items
total	INTEGER	Total number of items in the session
difficulty	TEXT	Difficulty level at the time of the session

Table 8. Schema of the session_log table.

daily_plan

Column	Type	Description
id	INTEGER PK	Auto-increment primary key
plan_date	TEXT	Target study date
exam	TEXT	Exam type for the day
topic	TEXT	Topical category
exercise_type	TEXT	Question format
count	INTEGER	Target number of practice items
done	INTEGER	Completion flag (0 or 1)
note	TEXT	AI-generated rationale or suggestion

Table 9. Schema of the daily_plan table.

settings

Key	Value Description
difficulty	Current adaptive difficulty level (easy / medium / hard)
reminder_time	User-configured notification time in HH:MM format
reminder_enabled	Boolean flag — whether reminders are active
reminder_last_fired	Date of the last notification to prevent daily duplicates

Table 10. Keys stored in the settings table.

7.3 Persistence Mechanism

The database layer uses a context manager pattern: **connect()** opens a SQLite connection, sets **row_factory = sqlite3.Row** for dict-like access, commits automatically when the context exits normally, and closes the connection in all cases. No ORM is used, which keeps the code compact and auditable, at the cost of manual SQL strings.

7.4 Database Snapshot

The repository ships with a pre-populated **data/catalyst.db** that demonstrates realistic learner history, confirming that the persistence layer and session logging mechanisms are functional:

Table	Row Count	Interpretation
mistakes	6	Six recorded incorrect answers across practice sessions
session_log	2	Two completed practice sessions with scores and difficulty
daily_plan	7	One complete seven-day study roadmap
settings	1	One settings record (difficulty level)

Table 11. Row counts in the pre-populated database snapshot.

SECTION 8

Source Code Structure

8.1 catalyst_main.py — Entry Point

This is the application entry point. It manually loads environment variables from a `.env` file, then imports and calls `vocab_catalyst.app.run_app()`. The loader is deliberately minimal: it reads key-value lines, skips comments and blank lines, strips surrounding quotes, and places the values directly into `os.environ`.

8.2 vocab_catalyst/app.py — Orchestrator

This is the largest and most central file. The `CatalystApp` class acts as the full-system orchestrator and contains:

- GUI class construction and tab layout
- Practice generation and submission workflow
- Mistake Bank UI and analytics triggers
- Calendar and roadmap interface
- Reminder configuration and background threading
- Adaptive difficulty adjustment at session end
- Session finalization and score logging

8.3 vocab_catalyst/db.py — Persistence Layer

This module defines schema creation, insert/update helpers, query helpers, and settings accessors. The implementation is direct and lightweight, using the context manager pattern described in Section 7.3.

8.4 vocab_catalyst/models.py — Data Contracts

This module defines the typed data contracts exchanged between Gemini and the application. `ExercisePayload` and `RoadmapDay` each include a `from_dict()` factory method with validation logic so that malformed model outputs are rejected before they can corrupt the UI or the database.

8.5 vocab_catalyst/prompts.py — Prompt Engineering

This module centralizes all prompt templates, including formatting rules, JSON schema expectations, roadmap instructions, hint constraints, and Vietnamese explanation prompts. Centralizing prompts ensures consistency across feature calls and simplifies future maintenance.

8.6 vocab_catalyst/gemini_service.py — AI Client Wrapper

This module wraps the Gemini client and provides one convenience method per AI task. It also includes JSON parsing helpers that strip optional Markdown code fences before decoding, protecting the application against a common model formatting habit.

8.7 vocab_catalyst/constants.py — Domain Configuration

Constant	Values
Supported exams	SAT, IELTS
SAT topics	Abstract/Logic, Science + Social, Tone
IELTS topics	Environment, Education, Society, Technology
Exercise types	Multiple defined formats and labels
Difficulty levels	Easy, Medium, Hard

Table 12. Constants defined in constants.py.

8.8 Legacy Module: vocab_quiz_app.py

This module is an earlier monolithic version of the quiz application, preserved in the repository for reference. It contains prompt generation, response parsing, a simple GUI, answer handling, hint generation, and wrong-answer explanation — but lacks the Mistake Bank, roadmap planning, and reminder subsystem that characterize the current modular architecture.

■ Repository Hygiene

vocab_quiz_app.py and main.py are not part of the active application stack. They should be clearly documented as legacy or archived to avoid confusion during formal submission review.

SECTION 9

Implementation Details

9.1 Key Algorithms

JSON Validation and Two-Stage Parsing

Gemini responses are expected to adhere to strict schemas. A two-stage validation process protects the application from malformed content:

- **Stage 1 — Structure:** `parse_exercises_json()` requires `{"exercises": [...]};`
`parse_roadmap_json()` requires `{"days": [...]};`
- **Stage 2 — Content:** each array item is validated by the relevant dataclass constructor (`ExercisePayload.from_dict()` or `RoadmapDay.from_dict()`).

This design significantly reduces the risk of a poorly formed LLM response causing a runtime exception or incorrect data being rendered to the learner.

Adaptive Difficulty Selection

At the end of each session, `_adaptive_difficulty()` updates the stored difficulty level using a transparent rule-based policy:

Condition	Outcome
All answers correct AND session ≥ 2 items	Difficulty increases one step (e.g., easy $\rightarrow$ medium)
Accuracy $< 45\%$	Difficulty decreases one step (e.g., hard $\rightarrow$ medium)
Any other result	Difficulty remains unchanged

Table 13. Adaptive difficulty adjustment policy.

✓ Design Choice

A simple rule-based policy was chosen over a statistical model intentionally. The approach is transparent, easy to debug, stable under edge cases, and appropriate for a single-user desktop tool where over-engineering would reduce maintainability.

Mistake-Frequency Ranking

`mistake_word_frequencies()` groups all mistakes by the lowercase target word and counts occurrences. The resulting ranked list is injected directly into generation prompts, ensuring that the most persistently missed vocabulary items receive disproportionately more practice time.

Reminder Debouncing

The reminder background loop stores **reminder_last_fired** as a date string in the **settings** table. Before sending a notification, the loop checks whether today's date already matches the stored value. This prevents duplicate notifications even if the application remains open for multiple hours.

9.2 Business Logic Rules

- Only SAT and IELTS are supported; no other exams appear in the UI.
- Only predefined topic lists are available; free-form topic entry is not supported.
- Each exercise session operates on a finite, AI-generated item queue.
- Incorrect responses are persisted immediately before feedback is displayed.
- Completed sessions are logged with score, total count, and difficulty level.
- Roadmap generation always covers exactly seven consecutive days — partial roadmaps are rejected.
- Reminders are optional, user-configurable, and macOS-only.

9.3 API Integration Architecture

All Gemini integration is centralized in **GeminiService**. The model **gemini-2.5-flash** is used consistently across all tasks. Each use case has its own dedicated prompt template:

Method	AI Task
generate_exercises()	New vocabulary multiple-choice exercises
generate_relearn()	Review exercises seeded from mistake history
generate_roadmap()	Seven-day adaptive study plan
explain_wrong_vi()	Vietnamese wrong-answer explanation
hint_vi()	Vietnamese hint without answer disclosure
analyze_mistakes_vi()	Vietnamese mistake pattern diagnosis

Table 14. GeminiService methods and their AI tasks.

9.4 Error Handling

The current codebase includes practical but lightweight error handling. The approach is to fail gracefully and keep the application usable even under partial failures:

- A missing API key disables AI-driven features while retaining local features (Mistake Bank, Calendar).

- Gemini calls are wrapped in worker threads with exception handling.
- Invalid JSON or malformed structures produce empty result sets or visible error messages.
- Reminder notification failures are silently swallowed in the background loop.
- Reminder time input is minimally validated before being saved.

■ Known Gap

The current implementation does not include retry logic, exponential backoff, or structured logging. Adding these would improve robustness under transient API failures and simplify debugging in production use.

9.5 Configuration Management

Mechanism	Purpose
Environment variable GEMINI_API_KEY	Supplies access credentials to the Gemini API
Optional .env file	Loaded manually by catalyst_main.py at startup
SQLite settings table	Stores user-specific runtime preferences (difficulty, reminders)

Table 15. Configuration mechanisms used by the application.

This hybrid approach is appropriate for a desktop application where the same machine holds both the user's settings and the application data.

SECTION 10

User Interface

10.1 Tab Layout

The GUI is organized into four tabs, each serving a distinct functional area. The interface is built entirely with CustomTkinter and follows an event-driven model where long-running operations are dispatched to background threads.

Tab 1 — Luyện tập (Practice)

- Exam selector (SAT / IELTS)
- Topic and exercise type dropdowns
- Difficulty selector and question-count slider
- Generate button
- Question display area
- Radio-button answer selection
- Submit and Hint buttons
- AI feedback text box
- Session status line

Tab 2 — Mistake Bank

- Scrollable list of recent incorrect answers (reverse chronological order)
- Refresh list button
- Analyze mistakes with AI button
- Generate relearning exercises button
- AI analysis text box

Tab 3 — Lịch &行程 (Calendar & Roadmap)

- Month navigation controls
- Interactive calendar grid
- Task status label for the selected date
- Roadmap generation button
- Scrollable task list with per-task completion checkboxes

Tab 4 — Nhắc nhở (Reminders)

- Reminder explanation text
- HH:MM time input field

- Enable / disable checkbox
- Save settings button
- Test notification button

10.2 Typical User Workflow

A normal learner session follows this sequence:

1. Open the application.
2. Select exam, topic, exercise type, and difficulty.
3. Generate a practice set.
4. Answer questions one by one.
5. Review AI explanations for each incorrect answer.
6. Inspect the Mistake Bank for accumulated errors.
7. Generate a relearning set or a 7-day roadmap.
8. Configure and enable a daily reminder for habit formation.

SECTION 11

Testing and Results

11.1 Functional Testing

■ Testing Approach

The repository does not contain an automated test suite. Evaluation is therefore conducted through code inspection, runtime logic analysis, and examination of the pre-populated database snapshot. The code structure supports manual functional testing of the following scenarios:

- Starting the app with and without **GEMINI_API_KEY**.
- Generating exercises and verifying Gemini response parsing.
- Submitting correct and incorrect answers.
- Verifying that wrong answers appear in the Mistake Bank.
- Generating a 7-day roadmap and checking database insertion.
- Toggling a plan-day completion checkbox.
- Saving reminder settings.
- Triggering a macOS desktop notification.

11.2 Execution Scenarios

Scenario 1 — Missing API Key

If **GEMINI_API_KEY** is absent, the application still launches successfully. AI-based generation features are unavailable, but local features — the Mistake Bank and Calendar — remain fully functional. The GUI displays an appropriate warning.

Scenario 2 — Wrong Answer Submission

When the learner selects an incorrect answer, the following occurs atomically:

- The mistake is written to SQLite with full metadata.
- The Mistake Bank view is refreshed.
- A Vietnamese explanation is requested from Gemini (asynchronously).
- The session advances to the next question.

Scenario 3 — Roadmap Generation

When the learner requests a roadmap, the application validates that the AI response contains exactly seven day entries. If the length differs, the result is rejected entirely and the user sees an error message. No partial roadmaps are written.

Scenario 4 — Reminder Trigger

If reminder mode is enabled and the current local time matches the stored reminder time, the application sends a desktop notification once per day. The debounce mechanism prevents duplicate notifications even during extended application sessions.

11.3 Observed Results from Database Snapshot

The pre-populated database confirms that the persistence layer and session logging mechanisms are functional. The snapshot contains realistic learner history including SAT abstract-logic mistakes, session performance records, and a complete seven-day study plan — demonstrating end-to-end operation of all core features.

SECTION 12

Challenges and Solutions

Challenge	Solution Implemented
Unreliable LLM Output Model responses can deviate from expected schemas.	Strict prompt instructions, code-fence stripping, JSON parsing, dataclass validation, and length checks for roadmap responses.
GUI Responsiveness Network calls to Gemini introduce noticeable latency.	Background threading for all generation, explanation, hint, and roadmap requests, ensuring the interface never freezes.
Meaningful Mistake Review Mistake data must be used — not merely stored.	Mistake Bank frequencies are queried and summarized, then injected into relearning and roadmap prompts.
Balancing Simplicity and Adaptation The system should adapt without becoming opaque.	A simple, transparent rule-based difficulty adjustment policy replaces a heavyweight analytics pipeline.
Platform-Specific Notifications Desktop reminders are not cross-platform.	Notifications are macOS-only, implemented via osascript, and safely no-op on all other platforms.

Table 16. Engineering challenges and their implemented solutions.

SECTION 13

Limitations

■ Scope Note

The limitations listed below are observations from a technical audit of the repository state. They represent areas for improvement rather than critical defects in the current implementation.

#	Limitation	Impact
1	No automated test suite	Lower confidence in regression safety
2	Dependence on Gemini availability	Core features require external API access and a valid key
3	macOS-only reminder delivery	Desktop notifications non-functional on Windows and Linux
4	No authentication or multi-user support	Application is limited to one local learner
5	Weak semantic validation of AI content	Structure is validated; semantic correctness of exercises is not deeply verified
6	Bundled vocabulary JSON not used	data/vocabsat.json exists but is not consumed by the active runtime
7	Documentation drift	RUN_GUIDE.md references vocab_loader.py and VOCAB_SYSTEM.md — neither present in the source tree
8	Unused declared dependencies	numpy and sounddevice in requirements.txt add unnecessary install footprint

Table 17. Current limitations of the Vocab Catalyst codebase.

SECTION 14

Future Improvements

Improvement	Expected Benefit	Priority
Add automated unit and integration tests	Improves reliability and submission quality	High
Reuse vocabsat.json in the active pipeline	Enables offline vocabulary sourcing and richer exercises	High
Add retry and backoff for Gemini calls	Improves robustness under transient API failures	High
Remove unused dependencies (numpy, sounddevice)	Reduces install footprint and clarifies scope	Medium
Resolve documentation drift in RUN_GUIDE.md	Reduces confusion for new contributors	Medium
Add import/export tools for Mistake Bank	Enables backup and migration of study history	Medium
Support Windows/Linux notifications	Expands platform compatibility	Medium
Implement richer analytics dashboards	Makes learning trends easier to interpret	Low
Persist more detailed attempt history	Enables deeper longitudinal analytics	Low
Replace raw SQL with a repository layer	Improves maintainability as the schema grows	Low
Create a packaging/installer workflow	Makes the project easier to distribute	Low

Table 18. Proposed future improvements, ranked by priority.

SECTION 15

Conclusion

The Vocab Catalyst repository implements a coherent, AI-powered vocabulary tutor that successfully combines local persistence, structured prompt engineering, and a responsive desktop interface. The application demonstrates that a single-developer project can meaningfully integrate generative AI into an educational workflow while maintaining architectural discipline.

The strongest aspects of the design are the clear separation between GUI, database, prompt, and service layers; the use of typed dataclasses to validate AI output at runtime; and the practical learning loop that records mistakes and converts them back into future adaptive practice.

From a software engineering perspective, the project is especially effective as a compact capstone-style system: it demonstrates application architecture, event-driven UI design, schema-based persistence, adaptive learning logic, and external API integration — all within a maintainable codebase.

✓ Overall Assessment

Vocab Catalyst is a technically credible and educationally useful implementation of an adaptive vocabulary practice platform. The codebase is functional, conceptually coherent, and provides a solid foundation for future extension. The primary areas for improvement are the addition of an automated test suite and cleanup of stale repository artifacts.

SECTION 16

References

All references are to source files within the analyzed repository snapshot.

[1]	catalyst_main.py	Application entry point and .env loading logic.
[2]	main.py	Gemini API smoke-test script (outside the application stack).
[3]	vocab_quiz_app.py	Legacy standalone quiz prototype — monolithic architecture.
[4]	vocab_catalyst/app.py	Main application GUI class and workflow orchestration.
[5]	vocab_catalyst/db.py	SQLite schema definition and persistence helper functions.
[6]	vocab_catalyst/gemini_service.py	AI client wrapper and JSON parsing utilities.
[7]	vocab_catalyst/models.py	Dataclass-based data validation and type contracts.
[8]	vocab_catalyst/prompts.py	Prompt engineering templates for all AI task types.
[9]	vocab_catalyst/constants.py	Supported exams, topics, exercise types, and difficulty levels.
[10]	data/vocabsat.json	Static vocabulary dataset (790 entries) included in the repository snapshot.